

```

9  _global_
10 void histo_kernel(unsigned char *buffer,
11                  long size, unsigned int *histo) {
12     __shared__ unsigned int histo_private[256];
13     // warning: this will not work correctly if there are fewer than 256 threads!
14     if (threadIdx.x < 256)
15         histo_private[threadIdx.x] = 0;
16     __syncthreads();
17     int i = threadIdx.x + blockDim.x * blockDim.x;
18     // stride is total number of threads
19     int stride = blockDim.x * gridDim.x;
20     while (i < size) {
21         atomicAdd(&histo_private[buffer[i]], 1);
22     }
23     // wait for all other threads in the block to finish
24     __syncthreads();
25     if (threadIdx.x < 256) atomicAdd(&histo[threadIdx.x], histo_private[threadIdx.x]);
26 }

```

```

int MPI_Send(void *buf, int count, MPI_Datatype datatype,
int dest, int tag, MPI_Comm comm)
- Buf: Initial address of send buffer
- Count: Number of elements in send buffer (non-negative integer)
- Datatype: Datatype of each send buffer element
- Dest: Rank of destination (integer)
- Tag: Message tag (integer)
- Comm: Communicator (handle)

void data_server(unsigned int vector_size) {
int np, num_nodes = np - 1, first_node = 0, last_node = np - 2;
unsigned int num_bytes = vector_size * sizeof(float);
float *input_a = 0, *input_b = 0, *output = 0;

/* Set MPI Communication Size */
MPI_Comm_size(MPI_COMM_WORLD, &np);

/* Allocate input data */
input_a = (float *)malloc(num_bytes);
input_b = (float *)malloc(num_bytes);
output = (float *)malloc(num_bytes);
if(input_a == NULL || input_b == NULL || output == NULL) {
printf("Server couldn't allocate memory\n");
MPI_Abort(MPI_COMM_WORLD, 1);
}

/* Initialize input data */
random_data(input_a, vector_size, 1, 10);
random_data(input_b, vector_size, 1, 10);

/* Wait for compute to complete */
MPI_Barrier(MPI_COMM_WORLD);

/* Collect output data */
MPI_Status status;
for (int process = 0; process < num_nodes; process++) {
MPI_Recv(output, process * num_points / num_nodes,
MPI_REAL, process, DATA_COLLECT, MPI_COMM_WORLD, &status);
}

/* Store output data */
store_output(output, dimx, dimy, dimz);

/* Release resources */
free(input);
free(output);
}

```

```

int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
int source, int tag, MPI_Comm comm, MPI_Status *status)
- Buf: Starting address of receive buffer
- Count: Maximum number of elements in receive buffer (non-negative integer)
- Datatype: Datatype of each receive buffer element
- Source: Rank of source (integer)
- Tag: Message tag (integer)
- Comm: Communicator (handle)
- Status: Status object

/* Send data to compute nodes */
float *ptr_a = input_a;
float *ptr_b = input_b;

for (int process = 1; process < last_node; process++) {
MPI_Send(ptr_a, vector_size / num_nodes, MPI_FLOAT,
process, DATA_DISTRIBUTE, MPI_COMM_WORLD);
ptr_a += vector_size / num_nodes;
MPI_Send(ptr_b, vector_size / num_nodes, MPI_FLOAT,
process, DATA_DISTRIBUTE, MPI_COMM_WORLD);
ptr_b += vector_size / num_nodes;
}

/* Compute the partial vector addition */
for (int i = 0; i < vector_size; ++i) {
output[i] = input_a[i] + input_b[i];
}

/* Or, can offload to GPU here */
cudaMemcpy(&output, cudaMemcpy(), kernel launch, etc. */
MPI_Barrier(MPI_COMM_WORLD);

/* Send the output */
MPI_Send(output, vector_size, MPI_FLOAT,
server_process, DATA_COLLECT, MPI_COMM_WORLD);

/* Release memory */
free(input_a);
free(input_b);
free(output);
}

```

```

// Insert your GPU convolution kernel code here
int numFile_W = (W_out-1)/TILE_SIZE+1;
// int numFile_H = (H_out-1)/TILE_SIZE+1;
int idx = threadIdx.x, tidy = threadIdx.y;
int bidx = blockIdx.x, bidy = blockIdx.y, bidz = blockIdx.z;

int idx_tile = bidx;
int idx_feature = bidx;
int idx_image = bidz;

int tile_x = idx_tile * numFile_W, tile_y = idx_tile * numFile_W;
int idx_pixel = tile_x + TILE_SIZE * W_out + tile_y * TILE_SIZE;
int output_x = idx_pixel / W_out, output_y = idx_pixel % W_out;

output_x += tidy;
output_y += tidy; // shift

int input_x = output_x * S, input_y = output_y * S;
float sum = 0;

for (int cc = 0; cc < C; cc++)
for (int xx = 0; xx < K; xx++)
for (int yy = 0; yy < K; yy++){
sum += in_4d[idx_image, cc, input_x+xx, input_y+yy] * mask_4d[idx_feature, cc, xx, yy];
}

if(output_x < H_out && output_y < W_out)
out_4d[idx_image, idx_feature, output_x, output_y] = sum;

#ifdef MC_4d
#endif out_4d
#ifdef in_4d
#endif mask_4d

// Set the kernel dimensions and call the kernel
int W_out = (W-K)/S+1;
int H_out = (H-K)/S+1;
int W_grid = (W_out-1)/TILE_SIZE+1;
int H_grid = (H_out-1)/TILE_SIZE+1;
int numTile = W_grid*H_grid;
dim3 DimGrid(W_grid, H_grid, 1);
dim3 DimBlock(TILE_SIZE, TILE_SIZE, 1);

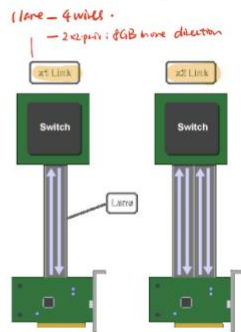
void convLayer_forward(int B, int M, int C, int H, int W, int K, float* X, float* W, float* Y) {
int H_out = H - K + 1;
int W_out = W - K + 1;

for (int b = 0; b < B; ++b)
for (int m = 0; m < M; ++m)
for (int h = 0; h < H_out; ++h)
for (int w = 0; w < W_out; ++w) {
Y[b, m, h, w] = 0.0f;
for (int c = 0; c < C; ++c)
for (int p = 0; p < K; ++p)
for (int q = 0; q < K; ++q)
Y[b, m, h, w] += X[b, c, h + p, w + q] * W[m, c, p, q];
}
}

```

PCIe Gen 3 Links and Lanes

- Each link consists of one or more lanes
 - Each lane is 1-bit wide (4 wires, each 2-wire pair can transmit 8Gb/s in one direction)
 - 2-wire pair is used for differential signaling
 - Upstream and downstream simultaneous and symmetric
 - Each Link can combine 1, 2, 4, 8, 12, 16 lanes-x1, x2, etc.
- Each byte data is 128b/130b encoded into 130 bits with equal number of 1's and 0's; net data rate 7.8768 GB/s per lane each way.
 - Thus, the net data rates are 985 MB/s (x1) 1.97 GB/s (x2), 3.94 GB/s (x4), 7.9 GB/s (x8), 15.8 GB/s (x16), each way



```

for (int b = 0; b < B; ++b)
for (int m = 0; m < M; ++m)
for (int h = 0; h < H_out; ++h)
for (int w = 0; w < W_out; ++w) {
Y[b, m, h, w] = 0.0f;
for (int c = 0; c < C; ++c)
for (int p = 0; p < K; ++p)
for (int q = 0; q < K; ++q)
Y[b, m, h, w] += X[b, c, h + p, w + q] * W[m, c, p, q];
}
}

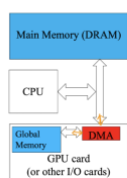
```

Current: 128/130 bit encoding

- Same goal: maintain DC balance while have sufficient state transition for clock recovery
- 1.5% overhead instead of 20%
- Scrambler function: long runs of 0s, 1s vanishingly small
- Instead of guaranteed run length of 8/10b
- At least one bit shift every 66 bits

PCIe Data Transfer using DMA

- DMA (Direct Memory Access) is used to fully utilize the bandwidth of an I/O bus
- DMA uses physical address (instead of virtual address)
- Transfers a number of bytes requested by OS
- Needs pinned memory



Pinned Memory

- DMA uses physical addresses
- The OS could accidentally page out the data that is being read or written by a DMA and page in another virtual page into the same location
- Pinned memory cannot be paged out
- If a source or destination of a cudaMemcpy in the host memory is not pinned, it needs to be first copied to a pinned memory - extra overhead
- cudaMemcpy is much faster with pinned host memory source or destination

```

global__ void spmvJDSKernel(float *out, int *matColStart, int *matCols,
int *matRowPerm, int *matRows,
float *matData, float *vec, int dim) {
//@@ insert spmv kernel for jds format
int idx = blockIdx.x * BLOCK_SIZE + threadIdx.x, iterate = 0;
float sum = 0;
if (idx < dim){
while(iterate < matRows[idx]){
sum += vec[matCols[matColStart[iterate]+idx]] * matData[matColStart[iterate]+idx];
iterate++;
}
out[matRowPerm[idx]] = sum;
}
return;
}

```

Allocate/Free Pinned Memory (a.k.a. Page Locked Memory)

- cudaHostAlloc()
 - Three parameters
 - Address of pointer to the allocated memory
 - Size of the allocated memory in bytes
 - Option - use cudaHostAllocDefault for now
- cudaFreeHost()
 - One parameter
 - Pointer to the memory to be freed

Using Pinned Memory

- The only difference is that the allocated memory cannot be paged by the OS
- The cudaMemcpy function should be about 2x faster with pinned memory
- Pinned memory is a limited resource whose over-subscription can have serious consequences